EduAmigo: testing report

What was tested, at how many users, what went wrong, and what was changed. Local testing on one MacBook, 25 October 2026.

In one paragraph. EduAmigo was tested with simulated users from 10 up to 1,000 at the same time, and every screen of every role was opened and used. The system stayed up and never showed one school's data to another. Errors were essentially zero: 4 requests out of about 124,000 failed in one Stage F run, and none failed in any other run. Testing found **one performance problem** (signing in was slow when about 1,000 people signed in within a minute, up to 4 seconds), which is now **fixed** (about 60 milliseconds), and **four real product bugs**, also fixed. **Not yet tested:** 2,500, 5,000 and 10,000 users, and several heavy features. Nothing here proves EduAmigo can carry 10,000 users.

1. How many users, and what happened

Simulated users signed in and then used the app the way a school would: 40 % students, 25 % parents, 20 % teachers, 15 % school office. Each paused 2 to 6 seconds between actions. Times are in milliseconds (ms); p95 means 95 out of 100 requests were faster than this.

Stage	Users	Requests	p50	p95	p99	Slowest	Errors	App CPU peak	DB connections	Outcome
A	10	555	4	13	74	108	0.00 %	16 %	28	Passed (first attempt gave a false pass; see section 3)
B	50	3,893	3	9	73	102	0.00 %	28 %	30	Passed (a false cross-school alarm on the first attempt; see section 3)
C	100	12,520	3	8	52	108	0.00 %	29 %	29	Passed
D	250	31,325	2	8	59	133	0.00 %	50 %	38	Passed
E	500	62,634	2	8	55	129	0.00 %	100 %	39	Passed
F	1,000	123,649	2	164	525	4335	0.003 %	117 %	82	Passed on the numbers, but sign-in was very slow: first problem found

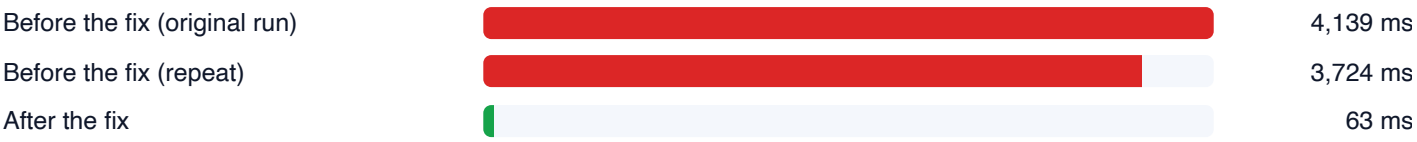
Target used: p95 at most 500 ms, p99 at most 1,000 ms, errors below 1 %, no sustained server errors, no data from one school visible to another. Stages G (2,500), H (5,000) and I (10,000) have **not** been run.

The one real performance problem: signing in at 1,000 users

At 1,000 users all signing in within about a minute (roughly 17 sign-ins per second), signing in took **about 4 seconds**, and ordinary pages also slowed to 3 seconds during that minute. Nothing failed, but it would feel broken.

Cause (measured, not guessed): passwords were checked by a library written in plain JavaScript that runs on the same single thread that answers every request. Each check takes about 50 ms, so 17 per second used most of that thread and everything else waited. The machine overall was only 11 to 17 % busy and the database was fine, so the limit was that one thread, not the hardware or MongoDB.

Fix: use the native version of the same password library, which does the checking on separate worker threads. Existing passwords keep working, nobody has to reset anything, and password strength was not reduced.



Stage F, same workload	Before the fix	Before the fix (repeat)	After the fix (final)
Sign-in p95 / p99	4,139 / 4,248 ms	3,724 / 3,935 ms	63 / 72 ms
Everything: p95	164 ms	156 ms	7 ms
Everything: p99	525 ms	443 ms	28 ms
Slowest single request	4,335 ms	4,019 ms	324 ms
Requests per second	299	301	304
Errors (HTTP)	0.003 % (4 requests)	0.00 %	0.00 %
App CPU average / peak	43 % / 117 %	41 % / 113 %	41 % / 127 %
MongoDB connections (peak)	82	87	44
Event-loop windows with p99 over 100 ms	not measured	21 of 211	0 of 212
Data checks and school separation	pass / pass	pass / pass	pass / pass

CPU use is the same before and after (about 41 % average): the work moved off the main thread, it did not disappear. Requests per second are the same because the simulated users pause for a fixed time, so this is **not** a sign of more capacity.

2. Product bugs found by testing (all fixed)

Where	Problem	Found by
Phone layout: Homework	The "Send homework" button was covered by the bottom navigation bar, so a teacher on a phone could not tap it.	Live multi-role test
Phone layout: Chat	The chat "Send" button was hidden behind the bottom navigation bar.	Live multi-role test
Student chat	Students saw a "Concerns" tab, which is only for parents and staff.	Live multi-role test
Office: Homework page	The office page asked the server for a "teacher profile". The office has none, so every visit produced a 403 error in the browser.	Page-by-page check (70 pages)

3. Problems in the tests themselves (found and corrected, so the numbers can be trusted)

- **A false pass.** The first Stage A run reported success, but the first 10 simulated users were all students, so parent, teacher and office traffic was never sent. The assignment was changed to spread roles evenly and the run was repeated.
- **A false alarm.** Stage B flagged "one cross-school response". Investigation showed the server answered 200 with an **empty** list and zero totals when asked for another school's student: no data from another school was exposed. The test now counts a response as a leak only when it contains records, and logs what it received. (It would be tidier for the server to answer 404 there.)
- Two functional checks were found to always report success, regardless of what happened. They were replaced with real checks before the results below were recorded.

4. Other testing done

Test	Result
Live multi-role test, on screen in four Chrome windows (admin, teacher, student, parent): notices, homework, attendance, fees, chat, concerns, profile photo, live bus, and a second school	**37 of 37 checks passed (after several mistakes in the test script itself were corrected). It confirmed that what one role saves reaches exactly the right people and nobody else.**
Page-by-page check: 70 pages, each at desktop and phone width	**No console errors, failed API calls, blank pages or sideways scrolling (after the one fix above).**
Permissions and saved data	**22 of 22. Student, parent and teacher are refused other roles' features and screens; a forged or missing sign-in is refused; a leave request, a notice and a new student are still there after a reload and after logging out and in.**
Backend automated tests	**67 test files, 1,294 tests, all passed (after the sign-in change).**
Frontend sign-in and workflow tests	**50 of 50 passed after the sign-in change. The last full frontend run (336 tests: 316 passed, 20 skipped by design) was before the sign-in change and was not repeated in full.**
Owner console automated tests	**24 of 24 (last run, earlier in the project).**

5. What was changed

In the product

- Sign-in now uses the native bcrypt password library (new file `src/services/passwordHash.js`, used by the user model and the password service). If it cannot load on some server, it automatically falls back to the old library, with one warning in the log.
- Sign-in concurrency limit: at most 3 password checks at the same time, up to 300 waiting. Beyond that a sign-in gets a clear "Sign-in is very busy, try again in a few seconds" message (503), never "wrong password". Reason: in a measurement, 250 sign-ins at once made an unrelated file read wait 3.4 seconds; with the limit it

waited under 1 millisecond. Cost: a pure sign-in flood is served about 30 % slower. Settings: `PASSWORD_CHECK_CONCURRENCY` and `PASSWORD_CHECK_QUEUE`.

- One new dependency: `bcrypt` (version 6) and two small helpers. Ready-made binaries are included for macOS, Linux and Windows.
- Frontend: the four fixes in section 2.

Not changed, on purpose

- No password was reset or re-hashed, the password strength setting (cost 10) was not lowered, who may sign in did not change, and the sign-in and password-reset rate limits are as before. No extra server instances were added.

In the testing tools (backend `loadtests/`)

- A one-command staged runner for Stages A to I. It uses a throw-away database on this Mac, never touches any real database, blanks every external-service key (no real email, SMS, payment or AI calls), and stops by itself on sustained server errors, any cross-school data, low memory, or if the app or database stops. Stages above 1,000 users need an explicit flag; Stage I also needs Stage H to have passed.
- Realistic user journeys for the four roles, after-run data checks, a cross-school check, a record of the test records created (with a separate, opt-in cleanup command), and an event-loop delay probe.

---

6. What is still unknown or unproven

- **Not run:** 2,500, 5,000 and 10,000 users. The load generator (k6) used about 3.4 to 5 GB of memory at 1,000 users, so 5,000 or more cannot be generated reliably from this Mac; a separate machine is needed.
- **Not in the load test yet:** fee payments, chat with attachments, concerns, profile-photo upload, the school bus GPS updates and live map, reports, year-end, and the AI features. Bus updates are likely the heaviest write load in real use.
- **The database in these tests is in memory on the same Mac.** A real hosted database over a network, with real data volumes, will behave differently. These figures show where the application itself becomes slow, not what production will do.
- Only one application instance was tested. The sign-in ceiling for one instance is now roughly 60 sign-ins per second; beyond that, more instances are the answer.
- One 361 ms pause was seen once in one run after the fix (120 ms in the other); its cause was not found.
- Browsers other than Chrome, and real phones, have not been tested.
- Live-test data on the local demo: the tests added notices, homework, attendance, a part-payment, chat messages and a bus trip to the local demo school only. Nothing was changed in any real school.
- Nothing has been committed, pushed or deployed. All changes are in the working folders.

---

7. Suggested next steps

1. Add fee, chat and bus-update journeys to the load test and re-run Stages B to F.
2. Run Stage G (2,500 users) after closing other apps; this needs an explicit go-ahead.
3. Rent a separate load-generator machine for 5,000 and 10,000 users, and point it at a test copy of the real hosting setup (same server size and database tier), not at production.
4. Before any of that goes live, review and commit the changes in groups.

Source files: backend `docs/production-readiness/LOAD_TEST_REPORT.md`, frontend `docs/production-readiness/LIVE_CHECK.md` and `FUNCTIONAL_CHECK.md`, and the run folders under backend `loadtests/results/`.